

Integrating GitHub Actions with Automated Unit Testing

1. Overview

In this Lab 7, you will learn how to integrate automated testing (unit and integration testing) into Github Actions. After this lab, you should be able to start incorporating automated testing into your team project.

2. Outcomes

Upon completion of this lab, you should be able to:

- Write test cases for automated unit testing using xUnit (JUnit, PHPUnit, PyUnit, etc) testing framework
- Create Github Actions to perform automated unit testing on a simple nodejs application
- Start incorporating Github Actions with automated testing into your team project
- Write test cases for automated integration and UI testing using Selenium
- Create Github Actions Workflow to perform automated integration and UI testing on a simple PHP application
- Start incorporating Github Actions Workflow into your team project

3. Server.js Code Reference

```
1  import express from 'express';
2
3  const app = express();
4
5  // Endpoint to return the current timestamp
6  app.get('/timestamp', (req, res) => {
7    res.json({ timestamp: getTimestamp() });
8  });
9
10 // Helper function to get the current timestamp
11 export function getTimestamp() {
12   return new Date().toISOString();
13 }
14
15 // Serve a simple HTML page
16 app.get('/', (req, res) => {
17   res.send(`
18     <!DOCTYPE html>
19     <html lang="en">
20     <head>
21       <meta charset="UTF-8">
22       <meta name="viewport" content="width=device-width, initial-scale=1.0">
23       <title>Browser Info</title>
24     </head>
25     <body>
26       <h1>Browser and Timestamp Info</h1>
27       <p id="browser-info">Loading browser details...</p>
28       <p id="timestamp">Fetching server timestamp...</p>
29       <script>
30         // Display browser information
31         const userAgent = navigator.userAgent;
32         document.getElementById('browser-info').textContent = 'Your browser: ' + userAgent;
33
34         // Fetch and display the server timestamp
35         fetch('/timestamp')
36           .then(response => response.json())
37           .then(data => {
38             document.getElementById('timestamp').textContent = 'Server timestamp: ' + data.timestamp;
39           })
40           .catch(err => {
41             document.getElementById('timestamp').textContent = 'Error fetching timestamp';
42           });
43       </script>
44     </body>
45     </html>
46   `);
47 });
48
49 // Start the server
50 const PORT = 3000;
51 const server = app.listen(PORT, '0.0.0.0', () => {
52   console.log(`Server is running on port ${PORT}`);
53 });
54
55 // Export the server for tests
56 export { server };
```

4. Pre-requisite

Git

Download the provided `github-actions-nodejs-test.zip` file which contains a simple node.js app, a corresponding unit testing file and selenium integration test file and a Github Actions Workflow yml test script template (located in `.github/workflows/` directory). I extracted the content to a folder name “`ssd-lab7`”. You can extract the zip file to your other lab directory.

Unzip and initialize it as a Git repository using the command:

```
git init
```

Next, commit with the commands:

```
git add .
```

then

```
git commit -m "Add initial files"
```

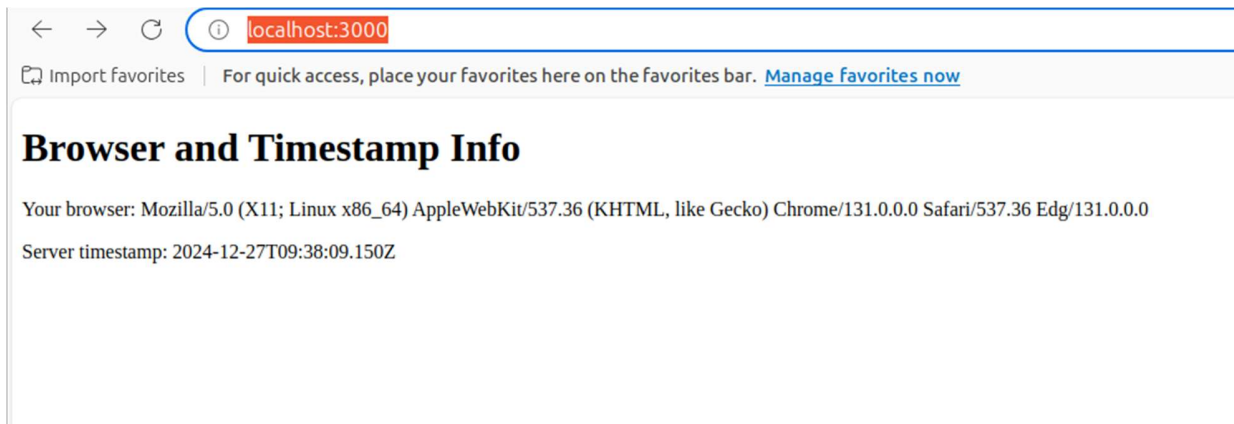
5. Run Server.js Locally

Start nodejs server

In the project root directory, run: `node src/server.js`

Open a browser, enter the url: <http://localhost:3000/>

You should see the following page:



6. Execute Unit Test Locally

Stop the node server.js using ctrl+c.

Review and understand the src code of tests/test.js.

You can run the unit test locally either using:

```
npx mocha tests/test.js
```

Or,

```
npm test
```

The definition of npm test script is found in package.json.

```
cloak@clkdev3 ~/Documents/SIT/ssd-labs-revision/ssd-lab7 (main)$npx mocha tests/test.js
Server is running on port 3000

Timestamp Function
  ✓ should return a valid ISO timestamp
  ✓ should return the current timestamp

2 passing (5ms)

cloak@clkdev3 ~/Documents/SIT/ssd-labs-revision/ssd-lab7 (main)$npm test
> test
> mocha tests/*.js

Server is running on port 3000

Timestamp Function
  ✓ should return a valid ISO timestamp
  ✓ should return the current timestamp

2 passing (3ms)
```

7. Set up Selenium Integration Test Locally

Recall that the Agile testing pyramid recommends not only automated unit testing, but also suitable amount of integration and UI testing although with lesser amount of automation.

You may be wondering how to perform automated integration and UI testing where user interactions are required. The answer lies with the use of headless browser which is a browser without GUI. Specifically, one of the popular tools available is Selenium coupled with WebDriver such as HtmlUnitDriver, FirefoxDriver or ChromeDriver, etc. In this lab, we will use Selenium together with ChromeDriver.

To help you, this Lab 7 will guide you on the use of Selenium to perform automated integration and UI testing so that you are able to start incorporating it into your team project.

Selenium automates browser testing or web app testing. In this lab, we are using selenium to test web application in a Chrome browser.

Depending on your development platform, you would need to install selenium server, selenium client web-driver and chrome driver.

For selenium server and web-driver installation, please refer to the instructions in:

- <https://www.selenium.dev/>
- Selenium download link: <https://www.selenium.dev/downloads/>
- Selenium API: <https://www.selenium.dev/selenium/docs/api/java/>
- HtmlUnit API: <https://www.selenium.dev/htmlunit-driver/index.html?org/opengaselenium/htmlunit/HtmlUnitDriver.html>

For chrome driver, please ensure the version is for your version of chrome driver.

- <https://developer.chrome.com/docs/chromedriver/get-started>
- Chrome driver download link:
- <https://googlechromelabs.github.io/chrome-for-testing/>

Alternatively, you can start a docker container with the necessary software installed. This is the method we would use for this lab:

```
docker run -d -p 4444:4444 --name selenium-server --network host selenium/standalone-chrome
```

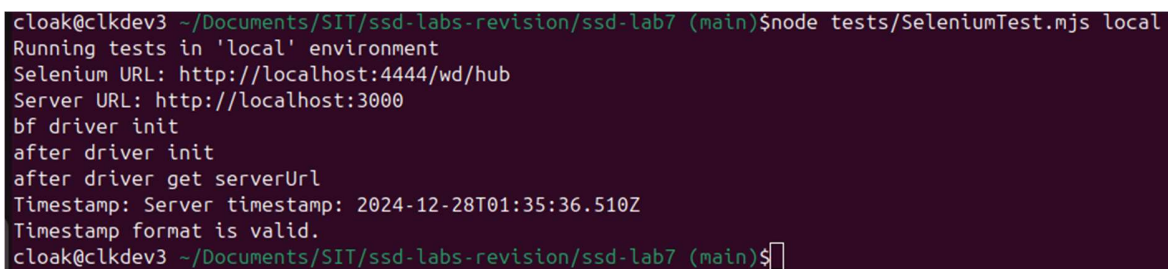
Check that the selenium server is running:

```
curl http://localhost:4444/wd/hub/status
```

8. Run Selenium Integration Test Locally

After setting up selenium and chrome driver, you can run the provided selenium test locally:

```
node tests/SeleniumTest.mjs local
```

A terminal window with a dark purple background showing the output of running the Selenium test. The text is as follows:

```
cloak@clkdev3 ~/Documents/SIT/ssd-labs-revision/ssd-lab7 (main)$node tests/SeleniumTest.mjs local
Running tests in 'local' environment
Selenium URL: http://localhost:4444/wd/hub
Server URL: http://localhost:3000
bf driver init
after driver init
after driver get serverUrl
Timestamp: Server timestamp: 2024-12-28T01:35:36.510Z
Timestamp format is valid.
cloak@clkdev3 ~/Documents/SIT/ssd-labs-revision/ssd-lab7 (main)$
```

9. Integrating Automated Unit and Integration Testing into Github Actions Workflows

Create new Github Actions Workflow

Github Action Workflow is configured mainly via yml files located under `.github/workflows/` directory.

A sample configuration file named `selenium-tests.yml` is provided. Please review its content.

Please also take time to read through the official github action documentation.

<https://docs.github.com/en/actions>

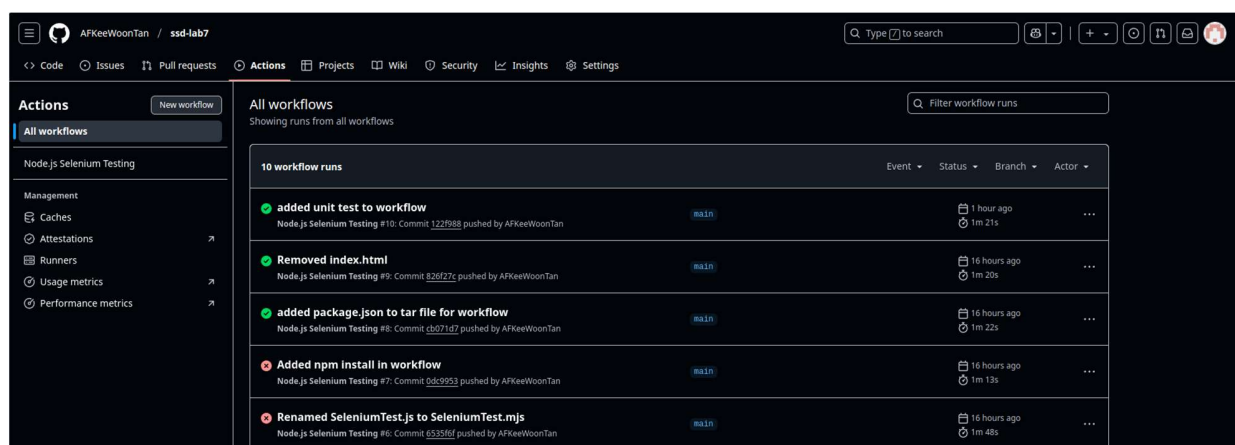
There are some important sections in the provided selenium-tests.yml file

1. The workflow is triggered based on the on: configuration in your test script. In the provided script, the workflow is triggered on pull or push requests on the main branch. You can change it to trigger only on push requests.
2. Please also note that the jobs section is divided into:
 1. a build section. This is for building the provided code. You will need to update this for your project.
 2. a test section. This is to set up the test environment. In this sample, a node image is used for the test container. And a selenium server service is started for testing. The build codes are transferred over for testing via a tar file. You will need to provide your own test configuration for your project.

Run Testing On Github Action Workflow

Commit and push your changes to Github.

Open a browser, and navigate to your Github action page.



Click on the workflow run, to view its status

The screenshot shows the GitHub Actions interface for a workflow named "added unit test to workflow #10". The workflow is in a "Success" state, triggered by a push to the "main" branch. The summary shows a total duration of 1m 21s and 1 artifact. The workflow file is "selenium-tests.yml" and it runs on push. The jobs are "build" (7s) and "test" (1m 0s). The annotations section shows 3 warnings: two about ubuntu-latest pipelines using ubuntu-24.04 soon, and one deprecation notice for artifact actions v1, v2, and v3.

Node.js Selenium Testing
added unit test to workflow #10

Summary

Jobs

- build
- test

Run details

- Usage
- Workflow file

Triggered via push 1 hour ago

AFKeeWoonTan pushed [122f988](#) [main](#)

Status: **Success** Total duration: **1m 21s** Artifacts: **1**

selenium-tests.yml
on: push

build 7s test 1m 0s

Annotations
3 warnings

- build**
ubuntu-latest pipelines will use ubuntu-24.04 soon. For more details, see <https://github.com/actions/runner-images/issues/10636>
- test**
ubuntu-latest pipelines will use ubuntu-24.04 soon. For more details, see <https://github.com/actions/runner-images/issues/10636>
- Deprecation notice: v1, v2, and v3 of the artifact actions**
The following artifacts were uploaded using a version of actions/upload-artifact that is scheduled for deprecation: "www-build". Please update your workflow to use v4 of the artifact actions. [Show more](#)

Click on build to view the build status

The screenshot shows the GitHub Actions interface for the same workflow, but with the "build" job selected. The job is in a "Succeeded" state, having run 1 hour ago in 7s. The job steps are listed with their durations: Set up job (1s), Checkout repository (0s), Cache Node.js modules (1s), Install Node.js dependencies (2s), Prepare artifacts (0s), Upload artifacts (0s), Post Cache Node.js modules (1s), Post Checkout repository (0s), and Complete job (0s).

Node.js Selenium Testing
added unit test to workflow #10

Summary

Jobs

- build
- test

Run details

- Usage
- Workflow file

Annotations
2 warnings

build
succeeded 1 hour ago in 7s

Search logs

- Set up job 1s
- Checkout repository 0s
- Cache Node.js modules 1s
- Install Node.js dependencies 2s
- Prepare artifacts 0s
- Upload artifacts 0s
- Post Cache Node.js modules 1s
- Post Checkout repository 0s
- Complete job 0s

Click on test to view the test status

The screenshot shows a GitHub Actions workflow run for a job named 'test'. The job has succeeded in 53s. The left sidebar shows the 'Summary' tab, with 'Jobs' listed as 'build' and 'test'. The 'Run details' section shows 'Usage' and 'Workflow file'. The main area displays a list of steps for the 'test' job, with the 'Run Selenium tests' step expanded to show its logs.

Step	Duration
> Set up job	0s
> Initialize containers	35s
> Download built files	1s
> Extract artifacts	0s
> Install dependencies	4s
> List node_modules	0s
> List files for debugging	0s
> Run unit tests	0s
> Start Node.js server	7s
> Verify server is running	0s
> Wait for Selenium server to be up	0s
> Run Selenium tests	1s
> Stop containers	1s
> Complete job	0s

Run Selenium tests

```
1 Run node tests/SeleniumTest.njs github
7 Running tests in 'github' environment
8 Selenium URL: http://selenium:4444/wd/hub
9 Server URL: http://testserver:3000
10 Before driver init
11 after driver init
12 after driver.get serverUrl
13 Timestamp: Server timestamp: 2024-12-28T02:45:49.755Z
14 Timestamp format is valid.
```

You can expand the section to view its details. The output depends on how you coded the test and the workflow config.

We hope that you have a better understanding of how to implement automated unit testing now. So try incorporating it into your team project. Enjoy!

END OF DOCUMENT